PostgreSQL-Backed Distributed Job Queue

Design, tradeoffs, implementation and interview prep: the full project, start to finish

The project in one sentence

A background job queue that uses PostgreSQL as the only coordination service. Async Python workers claim jobs with **SELECT ... FOR UPDATE SKIP LOCKED**, hold them with **renewable leases**, retry with delay, dead-letter jobs that keep failing, and expose Prometheus metrics. Batching claims and completions took throughput from about **1,100 to 17,800 jobs/s** (16 workers, no-op handlers) with **zero duplicate completions**.

Headline number	Value	Context
Sustained throughput	~17,800 jobs/s	16 workers, batch 50, 100k jobs, no-op handlers
Peak throughput	24-27k jobs/s	Same setup over 50k jobs
Speedup from batching	16x (16 workers), 2.9x (1 worker)	Before: about 9 DB round trips per job
Bulk enqueue	37-40k jobs/s	enqueue_many, 1,000 rows per call
Claim latency	p50 ~1.05 ms, p99 1.32-1.45 ms	Single-job claim
Crash recovery	5-6 s after SIGKILL	6 s lease, recovery interval 1 s
Duplicate completions	0	Across every benchmark run

Stack: Python 3.11, asyncio, asyncpg, PostgreSQL 16, Docker Compose, Prometheus, Grafana, Rich, GitHub Actions.

Contents

1. Pitches: 30 seconds and 2 minutes
2. Problem and goals
3. Architecture
4. Data model and job state machine
5. End-to-end job lifecycle (with the real SQL)
6. Key design decisions: why, alternatives, tradeoff, how
7. The performance story: finding and removing the bottleneck
8. Testing strategy and CI
9. Observability
10. Hardening: gaps found and fixed, and what remains
11. Scaling path
12. Interview questions with model answers (64)
13. Cheat sheet to memorize

How to use this document

Sections 1-9 explain what you built and why. Section 10 matters most for senior-level interviews: it covers the correctness gaps found in a self-review, how each was fixed and tested, and what is still open. Bring these up yourself. Section 12 has questions grouped by topic, with answers written the way you would say them.

1. Pitches

30-second version

"I built a distributed job queue on PostgreSQL to learn how systems like Celery or Sidekiq coordinate workers. Workers claim jobs atomically with SELECT FOR UPDATE SKIP LOCKED, so they never block each other or claim the same row. Each claim carries a time-limited lease, so if a worker crashes another worker picks the job up. Benchmarking showed the first version made about nine database round trips per job. I moved claim and completion into batched SQL functions, which made it 16 times faster: about 17,800 jobs per second with 16 workers, and no duplicate completions. I also wrote failure tests that SIGKILL a real worker process mid-job, and later added ownership checks so a stalled worker can't overwrite a job that was handed to someone else. All ten failure tests run in CI."

2-minute version (add these beats)

- **Why Postgres:** many apps already run it, so the queue adds no new infrastructure. You also get transactions, durability, and the ability to inspect the queue with plain SQL.
- **Semantics:** I was careful to call it at-least-once, not exactly-once. Locking stops two workers from claiming the same job at the same time. It can't stop a handler from running twice if a worker crashes after the side effect and before it records completion. Handlers need to be idempotent.
- **Reliability features:** lease renewal for long jobs, retries with a delay, dead-lettering, idempotency keys on enqueue, worker heartbeats, and an audit table of job events.
- **Performance insight:** the bottleneck was round trips, not lock contention. After batching, Postgres sat at about 470% CPU, and splitting workers across processes didn't change the numbers, so the database became the limit. The 100k run is about 30% slower than the 50k run. My hypothesis is dead index entries in the partial index, which I haven't verified yet.
- **Ops:** a Prometheus exporter, a Grafana dashboard and alert rules for queue depth, failure rate, no workers, and dead-letter growth.

2. Problem and goals

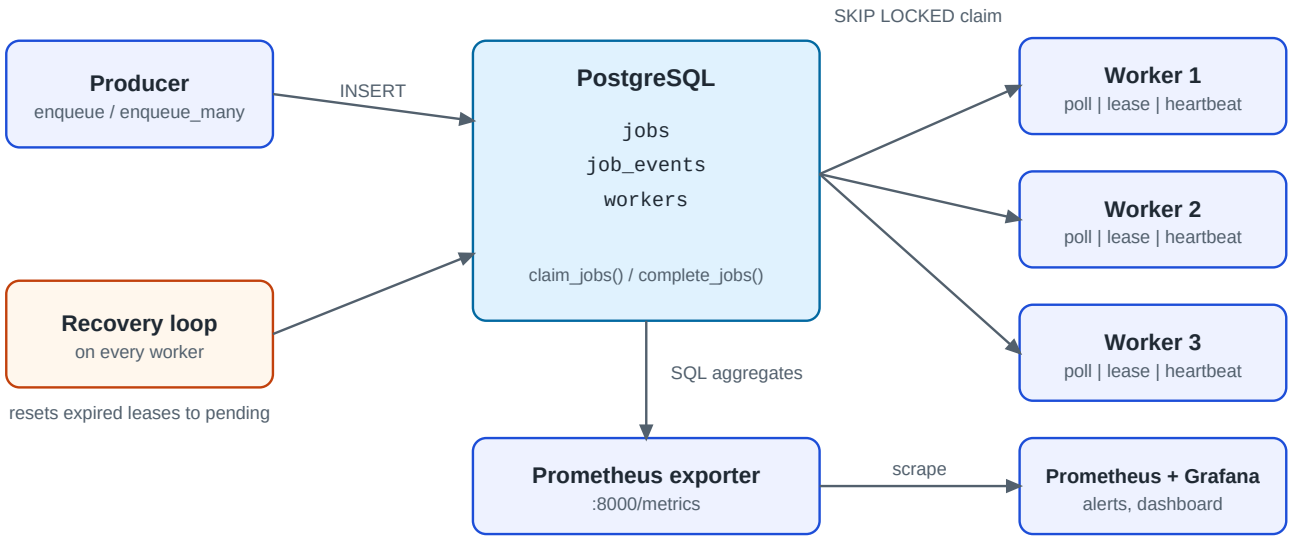
Web requests shouldn't block on slow work like sending emails, resizing images, generating reports or syncing with external APIs. That work goes to a background queue and a pool of workers. The hard part is **coordination**, not running the code:

- How do N workers pull from one queue without two of them taking the same job?
- What happens to a job when its worker dies halfway through?
- How do you retry temporary failures without retrying forever?
- How do you know the queue is healthy?
- When is Postgres good enough, and when do you need Kafka, SQS or Redis?

Goals and non-goals

Goals	Non-goals
Exclusive claiming of available jobs	Exactly-once side effects (impossible in general without idempotent handlers)
Recovery from worker crashes without manual work	Replacing Kafka/SQS at millions of messages per second
Retries, delays, dead-lettering, priority	Fan-out / pub-sub, message replay, long retention
Duplicate-safe job submission	Multi-region active-active
Measured performance and failure tests	Workflow DAGs, cron scheduling

3. Architecture



PostgreSQL holds all shared state: the jobs, their status, the leases, the worker registry and the event log. Workers are stateless. Any worker can die, and any other worker can pick up its expired work. Workers never talk to each other. All coordination goes through row locks and conditional UPDATEs in the database.

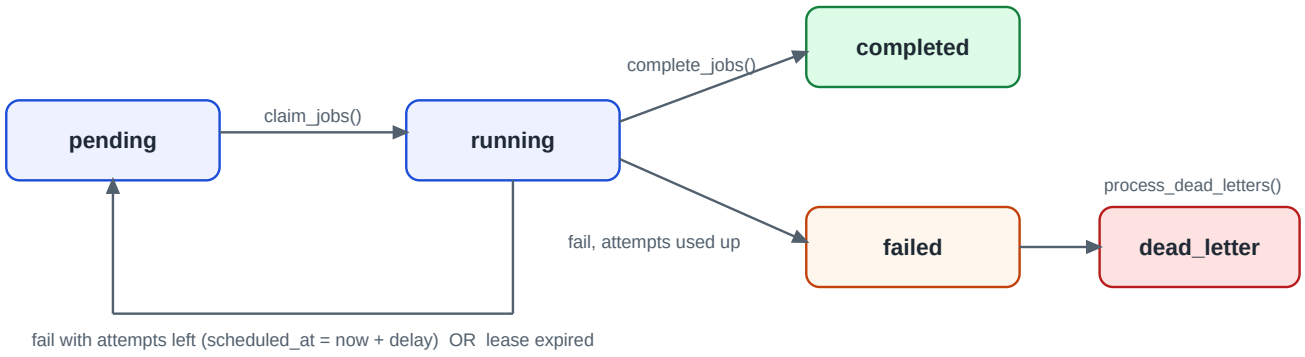
Components

File	Responsibility
`schema.sql`	Tables, partial indexes, and the stored functions claim_jobs, complete_jobs, renew_job_lease, recover_expired_leases, process_dead_letters
`queue_manager.py`	All SQL: enqueue, enqueue_many, claim_jobs, complete_jobs, fail_job, renew_leases, heartbeat, stats
`worker.py`	Three async loops per worker: poll/execute, heartbeat, recovery. Runs a batch concurrently with asyncio.gather and renews the batch lease in the background
`database.py`	asyncpg pool (min 2, max 10), JSONB codec, schema apply
`config.py`	Dataclass config with env overrides (BATCH_SIZE, NUM_WORKERS, DB_*)
`abstract_backend.py`	Backend interface (ABC) so the queue could later move to SQS/Kafka/Redis
`prometheus_exporter.py`	Standalone HTTP /metrics endpoint. Collects SQL aggregates every 15 s
`metrics.py, dashboard.py`	Snapshots to the metrics_snapshots table and a Rich terminal UI
`chaos_test.py`	6 failure-scenario tests, including a real SIGKILL of a worker process
`benchmark.py`	Enqueue rate, claim latency, multi-process throughput, duplicate detection
`docker-compose.yml`	Postgres, 3 worker containers (2 workers each), exporter, Prometheus, Grafana

Concurrency model

Each worker is an asyncio coroutine group. One OS process can run many workers, and they share one asyncpg connection pool. Handlers are **async** and I/O-bound, so concurrency comes from the event loop, not threads. For CPU-heavy handlers you would run multiple processes (the benchmark checks 4 processes x 4 workers) or push work to a process pool.

4. Data model and job state machine



jobs table (main columns)

Column(s)	Purpose
id UUID (gen_random_uuid)	Globally unique, created in the DB, safe to generate across many producers
queue_name, job_type, payload JSONB	Routing, handler lookup, arguments
priority INT (1 = highest)	Ordering at claim time
status job_status ENUM	pending / claimed / running / completed / failed / dead_letter
scheduled_at	Earliest time the job can run. Used for both delayed jobs and retry backoff
worker_id, lease_expires_at	Who owns the job, and until when
claimed_at, started_at, completed_at	Used for wait-time and execution-time metrics
attempt_count, max_attempts, retry_delay_sec	Retry policy, stored per job
last_error, last_error_at	Debugging failed jobs
idempotency_key TEXT UNIQUE	Deduplicates submissions
result JSONB, tags, metadata	Handler output and extra fields

Indexes and why each exists

Index	Serves
`idx_jobs_claimable (queue_name, priority, scheduled_at) WHERE status='pending'`	The hot claim query. A **partial** index stays small because it holds only pending rows, and its order matches ORDER BY priority, scheduled_at, so Postgres can walk the index and stop at LIMIT
`idx_jobs_lease_expiry (lease_expires_at) WHERE status IN (claimed, running)`	The recovery scan for expired leases
`idx_jobs_status (status, updated_at)`	Dashboards, archival
`idx_jobs_worker (worker_id)`	Finding a worker's jobs
`idx_job_events_job_id (job_id, occurred_at DESC)`	Per-job history, duplicate detection

Other tables

- **workers**: id, hostname, pid, status, last_heartbeat, jobs_processed/failed, current_job_id. Used for liveness and dashboards.
- **job_events**: append-only audit log (enqueued, claimed, started, completed, retried, failed...). It also serves as the source of truth for duplicate-completion checks.
- **metrics_snapshots**: periodic time-series rows for the terminal dashboard.

5. End-to-end job lifecycle

Step 1: Enqueue (idempotent)

```
INSERT INTO jobs (...) VALUES (...)
ON CONFLICT (idempotency_key) DO UPDATE SET updated_at = NOW()
RETURNING id, (xmax = 0) AS inserted;      -- xmax = 0 means a fresh insert, not an update
```

The same key returns the same job id. The `xmax = 0` trick tells you whether the row was inserted or already existed, so the 'enqueued' event is logged only once. Bulk path: `enqueue_many` passes parallel arrays through `unnest()` with `ON CONFLICT DO NOTHING`, in one round trip (37-40k jobs/s).

Step 2: Claim a batch (one statement, one round trip)

```
WITH picked AS (
    SELECT id FROM jobs
    WHERE queue_name = ANY($queues) AND status = 'pending' AND scheduled_at <= NOW()
    ORDER BY priority ASC, scheduled_at ASC
    LIMIT $limit
    FOR UPDATE SKIP LOCKED                  -- skip rows another worker has locked
), upd AS (
    UPDATE jobs j SET status='running', worker_id=$w,
           lease_expires_at = NOW() + make_interval(secs => $lease),
           claimed_at=NOW(), started_at=NOW(), attempt_count = j.attempt_count + 1
    FROM picked WHERE j.id = picked.id
    RETURNING ...
), ev AS (
    INSERT INTO job_events ... SELECT ... FROM upd   -- 'claimed' + 'started' events
)
SELECT ... FROM upd ORDER BY priority, scheduled_at;
```

Locking, state change and audit logging all happen in **one implicit transaction**. When this statement commits, the rows already say 'running' with this worker's id, so the row lock only lasts as long as the statement. The lease, not the lock, protects ownership after that. Time comes from the database (`NOW()`), so clock differences between workers don't matter.

Step 3: Execute concurrently, renew the lease

`_execute_batch` runs every handler with `asyncio.gather`. A background task calls `renew_leases` every 10 s (lease is 30 s), filtered by `worker_id = me AND status = 'running'`. A long job therefore keeps its lease, and a dead worker's lease runs out. `renew_leases` returns the ids it actually renewed. Any job missing from that set has been lost, so the worker cancels that handler and stops working on it.

Step 4: Complete in one round trip

`complete_jobs(worker, ids[], results[], durations[])` unnests the arrays, marks the rows completed, writes the completed events and bumps the worker's counter, all in one statement. The UPDATE only matches rows where `worker_id = p_worker_id AND status = 'running'`, so a worker that lost its lease can't complete the job. The function returns how many rows it completed, and the worker logs any rejected completions.

Step 5: Failure, retry, dead letter

`fail_job` is one statement: an UPDATE with CASE on attempt_count, the same ownership check as completion, and the event INSERT in a CTE.

- If attempt_count < max_attempts: set status back to pending, store last_error, and set `scheduled_at = NOW() + retry_delay`. The claim query ignores the job until then. The delay reuses the same scheduling column.
- If attempts are used up: status = failed. The recovery loop's `process_dead_letters()` then moves it to dead_letter.
- A missing handler for the job_type counts as a failure, so it is retried and then dead-lettered instead of blocking the queue.

Step 6: Crash recovery

Every worker runs `recover_expired_leases()` every 15 s. It is one UPDATE that resets any claimed/running job whose lease has expired back to pending. If the expired attempt was the last one allowed, the job goes to **dead_letter** instead, so a job that crashes its worker every time can't loop forever. Each recovery writes a lease_expired or dead_lettered event. Running it from several workers at once is safe: the inner SELECT uses FOR UPDATE SKIP LOCKED, so each job is reset once. The next claim increments attempt_count, which the SIGKILL test checks (attempt_count == 2).

6. Key design decisions

For each decision: why it was made, what else was considered, what was given up, and how it is implemented in code.

6.1 PostgreSQL as the queue (instead of Redis, RabbitMQ, Kafka or SQS)

Why this choice	The app already depends on Postgres, so the queue needs no new infrastructure. It gives ACID transactions, durability, row-level locks, indexes, and a queue you can inspect with SQL ('show every failed job of type X in the last hour'). You can also enqueue a job **in the same transaction** as a business write (the transactional outbox pattern).
Alternatives considered	Redis lists/streams (fast, but weaker durability and you run another service). RabbitMQ (real broker with acks, another service to run). Kafka (very high throughput and replay, heavy to operate, partitions limit parallelism). SQS (managed and unlimited, but at-least-once, no SQL, vendor lock-in).
Tradeoff accepted	Queue traffic shares connections, CPU and WAL with application queries. Polling adds load. MVCC means every state change writes a new row version, which creates dead tuples, bloat and vacuum pressure. It doesn't scale out horizontally the way a partitioned log does.
How it was achieved	One jobs table plus status enum, partial indexes, PL/pgSQL and SQL functions. AbstractQueueBackend keeps the option to change backends later.

6.2 SELECT ... FOR UPDATE SKIP LOCKED for claiming

Why this choice	Workers never wait on each other. If worker A has locked rows 1-10, worker B's query skips them and takes 11-20. That avoids head-of-line blocking, and it is the pattern used by Que, GoodJob, Oban, Solid Queue and pg-boss.
Alternatives considered	Plain FOR UPDATE: workers queue up behind the same top row. Optimistic UPDATE ... WHERE status='pending' with retry: many conflicts under load. Advisory locks: work, but need careful cleanup. A central dispatcher process: a single point of failure and a bottleneck.
Tradeoff accepted	Ordering is **approximate**: a locked high-priority row is skipped, so a lower-priority row may be taken first. No strict FIFO across workers. Works best when the claim transaction is short.
How it was achieved	`claim_jobs()` CTE: SELECT ... ORDER BY priority, scheduled_at LIMIT n FOR UPDATE SKIP LOCKED, then UPDATE to running in the same statement.

6.3 Leases instead of holding a transaction open while the job runs

Why this choice	Holding a transaction (and its row lock) for the whole handler is simple, but it pins a connection per running job, leaves long-running transactions that block vacuum, and gives no visibility. A lease is a timestamp in a row: the lock is released right away and ownership is a piece of data.
Alternatives considered	Long transaction holding the lock until done. Heartbeat-only liveness (decide a worker is dead from the workers table, then release its jobs). A visibility timeout with no renewal (SQS-style).
Tradeoff accepted	Recovery speed versus false recovery. A short lease recovers quickly but may reclaim a job whose worker is only paused (GC pause, network hiccup), which runs it twice. A long lease is safer but slower to recover. Defaults: 30 s lease, renewed every 10 s, so a worker can miss two renewals before losing the job.
How it was achieved	`lease_expires_at` set at claim, `renew_leases` (filtered by worker_id + status) runs in a background task, and `recover_expired_leases()` runs on every worker. Completion, failure and renewal all check ownership, so a worker that resumes after its lease expired can't overwrite the new owner.

6.4 Batched claim and completion (the 16x win)

Why this choice	Profiling showed about 9 round trips per job (claim, 3 event inserts, mark running, 2 heartbeats, complete, stats). With no-op handlers, network round trips and per-statement overhead dominated. Batching spreads the fixed cost over N jobs.
Alternatives considered	Pipelining individual statements. Prepared statements alone. More workers (helped about 7x but plateaued near 1.1k/s).
Tradeoff accepted	**Latency for throughput.** One lease covers the whole batch, and the results are written only after every job in the batch finishes, so one slow job delays completion of the fast ones. A large batch can also starve other workers when the queue is small. Mixed-latency job types should use separate queues or small batches.
How it was achieved	SQL functions with data-modifying CTEs (`claim_jobs`, `complete_jobs`) that take arrays through `unnest`. `BATCH_SIZE` is set from the environment.

6.5 At-least-once delivery, and saying so

Why this choice	Exactly-once execution of side effects is impossible in a distributed system where the worker can crash between 'do the effect' and 'record that it was done'. Claiming that guarantee would be wrong.
Alternatives considered	Exactly-once claims (an early scale_story.md table said this; now corrected, see Section 10). At-most-once (mark done before running, which loses jobs on crash).
Tradeoff accepted	Handlers must be idempotent: use deduplication keys at the downstream system, upserts, or check-then-act inside the same transaction as the effect.
How it was achieved	README states the semantics. Idempotency keys cover duplicate *submission*. Benchmark checks for duplicate *completions* through job_events.

6.6 Idempotency keys on enqueue

Why this choice	Producers retry too (an HTTP timeout after a successful insert). A UNIQUE key makes 'submit job' safe to retry.
Alternatives considered	Deduplicate in the app with a SELECT first (racy). Redis SETNX with a TTL (another service).
Tradeoff accepted	Keys are unique forever (no TTL), so the table grows. ON CONFLICT DO UPDATE writes a row version even for duplicates (cost). It doesn't stop the handler from running twice.
How it was achieved	`idempotency_key TEXT UNIQUE` + `ON CONFLICT ... RETURNING (xmax = 0)`; bulk path uses DO NOTHING.

6.7 Polling (instead of LISTEN/NOTIFY)

Why this choice	Simple and robust: no lost notifications, works through PgBouncer in transaction mode, and does nothing extra when the queue is busy because the worker loops straight back to claim.
Alternatives considered	LISTEN/NOTIFY wakeups (lower latency when idle, but you still need polling as a fallback because NOTIFY can be missed while disconnected). Long-poll via advisory locks.
Tradeoff accepted	Idle workers run one query every poll_interval (1 s), so pickup latency is up to 1 s when idle and there is a constant load of about N queries per second.
How it was achieved	`_poll_loop`: claim, and if the result is empty, sleep(poll_interval_sec). Listed as future work: LISTEN/NOTIFY.

6.8 asyncio + asyncpg

Why this choice	Background jobs are mostly I/O-bound (SMTP, HTTP, S3). asyncio runs many concurrent handlers per process cheaply. asyncpg is one of the fastest Postgres drivers for Python (binary protocol, prepared statements).
Alternatives considered	Threads + psycopg2 (GIL, heavier). multiprocessing only. Celery-style prefork.
Tradeoff accepted	CPU-heavy handlers block the event loop. A blocking library inside a handler stalls every job in that process. Needs async-aware handler code.
How it was achieved	Workers are coroutines sharing one pool. Benchmark checks that 1 process x 16 and 4 processes x 4 perform the same.

6.9 Business logic in SQL functions

Why this choice	One round trip, atomic, and the database plans it close to the data. Every client (Python or not) gets the same semantics.
Alternatives considered	Explicit multi-statement transactions from Python (more round trips).
Tradeoff accepted	Logic is split between Python and SQL. Harder to unit test and version (no migrations tool here, it is CREATE OR REPLACE).
How it was achieved	`claim_jobs`, `complete_jobs` (LANGUAGE sql), plus PL/pgSQL `renew_job_lease`, `recover_expired_leases`, `process_dead_letters`. The schema is applied on startup and is safe to re-run.

6.10 Database time, not worker clocks

Why this choice	Leases compare against `NOW()` on one server, so clock skew between worker hosts doesn't matter.
Alternatives considered	Timestamps from the worker (wrong across hosts).
Tradeoff accepted	Note that NOW() is the transaction start time, which is fine for short statements.
How it was achieved	Every lease and schedule calculation happens in SQL.

6.11 Prometheus exporter as a separate service

Why this choice	Metrics come from the database (the source of truth), so one exporter process covers every worker. There's no need to aggregate per-worker counters.
Alternatives considered	Embed a prometheus_client in every worker and use push gateway or per-pod scrape.
Tradeoff accepted	Aggregate queries (COUNT FILTER, PERCENTILE_CONT) scan the jobs table every 15 s, which gets expensive as it grows. Values come from table state, not true monotonic counters.
How it was achieved	stdlib HTTPServer on a thread + an asyncio collection loop. Alert rules for depth, failure rate, no workers, DLQ growth, p99.

7. The performance story

Interviewers often ask for an optimization you made. This is the strongest one in the project.

Method

- Benchmark against real Postgres 16 (Docker, 6 vCPU, fsync and synchronous_commit **on**). No-op handlers isolate queue overhead.
- Fill the queue first, start the clock at the first claimed job (excludes process spawn time), stop when every job is completed.
- Check correctness on every run: count jobs with more than one 'completed' event.
- Run twice to check variance. Try different worker counts, batch sizes and process layouts.

Diagnosis

Version 1 made about **9 round trips per job**. 1 worker gave 162 jobs/s, about 6 ms per job, which is roughly 9 x 0.6 ms. Sixteen workers reached only about 1,100 jobs/s. The cost per job was round trips, not lock waits.

Fix and result

Workers	Batch	Jobs	Throughput (run 1 / run 2)	Duplicates
1	1	2,000	471 / 464	0
16	1	10,000	2,069 / 2,021	0
16	10	20,000	13,159 / 13,185	0
16	50	50,000	26,610 / 24,174	0
16 (4 proc x 4)	50	50,000	26,327 / 26,019	0
16	50	100,000	17,779 / 17,778	0

What the numbers say

- **Python isn't the bottleneck:** 1 process vs 4 processes gives the same throughput, and Postgres was at about 470% CPU, so the limit is the database.
- **Longer runs are slower (about 30%):** hypothesis: completed rows leave dead tuples / dead index entries in the partial pending index until autovacuum runs, so later claims scan more. *Not verified.* The next step would be EXPLAIN (ANALYZE, BUFFERS) early vs late in a run, pg_stat_user_tables.n_dead_tup, and aggressive per-table autovacuum or partitioning.
- **Batch size has diminishing returns:** going from 1 to 10 gave about 6.5x, and 10 to 50 about 2x.
- Say clearly that these are **overhead numbers**. Real handlers do I/O, and handler time would dominate.

8. Testing strategy and CI

Scenario	What it checks	How
Idempotency	Same key gives the same id	3 enqueues with one key, compare the ids
Exclusive claiming	20 jobs, 5 concurrent workers, each completed exactly once	Count 'completed' events per job == 1
Dead letter	Always-failing job stops after max_attempts with its error stored	max_attempts=2, poll status
Priority	HIGH, MED, LOW order with one worker	Enqueue in reverse order, record execution order
Lease renewal	25 s job with a 30 s lease and 10 s renewal completes	Check status after 40 s
Crash recovery	SIGKILL a worker **OS process** mid-job, and another worker finishes it	multiprocessing spawn, lease 6 s, kill, check completed and attempt_count == 2 (5-6 s)
Fencing	A stale worker can't renew, complete or fail a reclaimed job, and the owner's result is the one stored	Claim with 1 s lease, expire, recover, re-claim as B, then A tries all three writes
Poison pill	A job that loses its worker on every attempt ends in dead_letter, with events logged	max_attempts=2, claim and let the lease expire twice, check status and event sequence
Lost lease in a batch	Renewal notices a stolen job and cancels only that handler	Batch of 2, reassign one in the DB mid-run, check it is untouched and the other completes
Batch lease hold	A finished job keeps its lease until the slow rest of its batch is recorded	0 s + 5 s jobs, 2 s lease, recovery every 0.3 s; fast job completes on attempt 1

Why SIGKILL in a separate process matters: cancelling an asyncio task runs `finally` blocks and cleanup code. SIGKILL can't be caught, so it is the closest thing to a real crash (OOM kill, node loss). The test proves recovery works *without* any cooperation from the dying worker.

CI: GitHub Actions starts a postgres:16-alpine service container and runs all 10 scenarios in `chaos_test.py` on every push to main and every PR. A non-zero exit fails the build.

Honest gaps: no unit tests per function, no property-based or Jepsen-style tests, no tests for graceful shutdown or network partitions. The stalled-worker case is simulated with a short lease, not a real SIGSTOP of a process. The priority test uses one worker, so it doesn't test global ordering.

9. Observability

Signal	Metric	Alert
Backlog	jobqueue_pending_total	QueueDepthHigh > 500 for 2 min, Critical > 2000
Throughput	jobqueue_throughput_per_minute	(dashboard)
Latency	avg wait, avg/p95/p99 exec seconds	HighExecutionLatency p99 > 30 s
Errors	jobqueue_failure_rate, dead_letter_total	Failure rate > 5%, DLQ +10 in 10 min
Capacity	jobqueue_active_workers (heartbeat < 30 s)	NoActiveWorkers == 0, WorkerCountLow < 2
Per-worker	jobs_processed / failed / alive	(dashboard)

The golden signals map to this queue as follows. **Saturation** is queue depth and wait time. **Errors** are failure rate and DLQ growth. **Latency** is execution percentiles. **Traffic** is throughput. The job_events table adds a

per-job audit trail for debugging ('why did job X run three times?').

10. Hardening: gaps found and fixed, and what remains

> **Why this section matters**
> Strong candidates know where their own system breaks. A self-review of the code found four correctness gaps and an overclaiming doc. All five are now fixed, and each code fix has a failure test in CI. Telling this story ('I reviewed my own system, found X, fixed it, and wrote a test that would have caught it') is usually more convincing than the original feature list.

10A. Fixed

10.1 No ownership check on completion (stalled / zombie worker)

Problem: `complete_jobs` and `fail_job` updated by job id only. Worker A pauses past its lease, recovery hands the job to worker B, and A wakes up and marks it completed (two completed events) or resets B's running job to pending.

Fix: Every terminal write now requires `worker_id = me AND status = 'running'` (renew already did). `complete_job`/`fail_job` return False and `complete_jobs` returns a count, so the worker knows a write was rejected.

Test: zombie_fencing: A claims with a 1 s lease, it expires, B reclaims; A's renew, complete and fail are all rejected; B completes once with B's result.

10.2 Batch lease renewal ignored lost leases

Problem: `_batch_lease_renewal_loop` threw away the row count from `renew_leases`, so a worker kept running jobs it no longer owned.

Fix: `renew_leases` now returns the ids it renewed (UPDATE ... RETURNING id). Each handler runs as its own task, and the loop cancels the task for any job missing from that set and stops renewing it. Jobs whose handler already finished keep being renewed until the batch's completions are written; an early version of this fix forgot that, and a test now covers it.

Test: lost_lease: a batch of 2, one job reassigned in the DB mid-run; only that handler is cancelled, the job is left alone, the other completes. **batch_lease**: a 0 s job and a 5 s job share a batch with a 2 s lease while recovery runs constantly; the fast job completes on attempt 1 with no lease expiry.

10.3 Poison pill retried forever

Problem: `recover_expired_leases()` reset to pending without checking attempts, so a job that crashes its worker every time (OOM, segfault) looped forever.

Fix: Recovery sets `CASE WHEN attempt_count >= max_attempts THEN 'dead_letter' ELSE 'pending'`, stores a last_error, and logs a lease_expired or dead_lettered event (this also closes the missing recovery audit trail). The inner SELECT uses FOR UPDATE SKIP LOCKED.

Test: poison_pill: max_attempts=2, the lease expires twice; the job ends in dead_letter with events [lease_expired, dead_lettered].

10.4 Non-atomic fail_job

Problem: It was a SELECT followed by a separate UPDATE and a third event INSERT: a read-then-write race and three round trips.

Fix: One statement: UPDATE with CASE on attempt_count, ownership check, and the event INSERT in a CTE (same style as complete_jobs).

Test: Covered by dead_letter and zombie_fencing.

10.5 Overclaims in scale_story.md

Problem: Its comparison table said 'Exactly-once: YES' and '~10k/min max'. Lock-contention latencies were labelled 'measured' but didn't come from any benchmark.

Fix: Changed to at-least-once, used the benchmarked 17.8k jobs/s, labelled the remaining thresholds as estimates, and corrected the connection math (the pool is per process, not per worker).

Test: n/a (documentation)

10B. Still open (know these and your fix)

10.6 Same-worker reclaim (full fencing token)

Problem: The ownership check compares worker_id. If the *same* worker id happens to reclaim the job, a stale write from its earlier attempt would still match.

Fix: Add a claim_version column that increments on every claim and must match on complete, fail and renew. This is the textbook fencing token.

10.7 Retry delay is fixed, with no exponential backoff or jitter

Problem: Every retry waits retry_delay_sec. A downstream outage produces synchronized retry storms.

Fix: `scheduled_at = NOW() + delay * 2^(attempt-1) * random(0.5..1.5)`, capped.

10.8 Whole-batch latency coupling

Problem: Completions are written only after `gather` finishes, so one slow job delays completion records for fast ones. Priority is respected only at claim time, because jobs in a batch run concurrently.

Fix: Complete each job as it finishes (or in micro-batches with a flush timer). Release leases one by one. Use separate queues by latency class.

10.9 Graceful shutdown

Problem: Cancelling worker tasks leaves claimed jobs 'running' until the lease expires, which delays them by up to 30 s and counts as an extra attempt. `stop()` deletes the worker row but doesn't release its jobs.

Fix: On SIGTERM: stop claiming, drain in-flight jobs with a timeout, then set remaining owned jobs back to pending without counting an attempt.

10.10 Table growth and bloat

Problem: Completed rows and job_events are never deleted. The exporter's COUNT/PERCENTILE queries scan the whole queue's history. This likely explains the slowdown on the 100k run.

Fix: Retention job (archive/delete), time-partitioned job_events with DROP PARTITION, per-table autovacuum settings, and maintained counters instead of COUNT(*).

10.11 Exporter 'counters' are not counters

Problem: `jobqueue_completed_total` is the current row count, which drops if rows are archived. Prometheus rate()/increase() treats a drop as a counter reset.

Fix: Expose them as gauges, or keep true monotonic counters (for example from job_events or in-process prometheus_client).

10.12 Smaller issues

Problem: f-string SQL in older tests and the benchmark (test-only, but reviewers will notice). The 'claimed' status and claim_next_job / mark_running are legacy paths the batch path no longer uses. Idempotency keys never expire. Grafana image uses :latest. Default admin/admin credentials. No schema migrations tool.

Fix: Parameterize queries, delete dead code paths, add key TTL / scoping, pin images, use Alembic or sqitch.

11. Scaling path

Limit hit	Symptom	Next step
Connections	TooManyConnections, slow pool acquire	PgBouncer in **transaction** mode (works because claims are single statements and there's no LISTEN). Smaller per-process pools
Claim hot spot / CPU	DB CPU pegged, claim p99 rising	Bigger batches, partition jobs by queue_name (LIST), dedicated DB for queue traffic, faster storage
Bloat	Throughput falls over a long run, index grows	Aggressive autovacuum per table, retention/archival, time partitions you can drop
Idle polling load	Many queries per second with an empty queue	LISTEN/NOTIFY wakeups + slower fallback poll, jittered backoff
Single primary / HA	Outage = queue down	Streaming replication + Patroni, or RDS/Cloud SQL Multi-AZ. Workers retry on failover
Beyond one Postgres	Write throughput ceiling	Move to SQS, Kafka or Redis Streams behind AbstractQueueBackend. Shard queues across DBs

Rule of thumb you can defend: Postgres queues work well up to thousands of jobs per second with real handlers, especially when transactional enqueue matters. Move to a broker when you need fan-out, replay or retention, very high throughput, or independent scaling and isolation from the main database.

12. Interview questions with model answers

A. Project overview and motivation

Q1. Walk me through the project.

Use the 2-minute pitch in Section 1. Order: problem, Postgres plus SKIP LOCKED, leases, retries/DLQ, the performance work with numbers, chaos tests in CI, and what you'd do next.

Q2. Why build this instead of using Celery?

To learn the coordination mechanisms underneath: claiming, leases, recovery, delivery semantics. In production I'd usually pick an existing tool (Celery, Oban, GoodJob, pg-boss, SQS). Building one taught me how to judge those tools: what their visibility timeout means and why handlers must be idempotent.

Q3. What was the hardest part?

Getting the semantics right, not the code. Realizing that exclusive claiming is different from exactly-once execution, then designing leases and recovery so a crash never loses a job, accepting that it may run a job twice.

Q4. What would you do differently?

Design ownership checks into every write from day one; I added them after a self-review found that a stalled worker could overwrite a reclaimed job. I'd also complete each job as it finishes instead of per batch, add a retention policy early, and benchmark with realistic handlers and multiple hosts. (See Section 10.)

Q5. Who is the user of this system and what workload is it for?

An application team that already runs Postgres and has moderate background work: emails, reports, image processing, syncs. Hundreds to a few thousand jobs per second, where transactional enqueue and SQL visibility matter more than maximum throughput.

B. Concurrency and locking

Q6. How do you guarantee two workers don't take the same job?

`SELECT ... FOR UPDATE SKIP LOCKED` inside the claim statement. The first worker locks the rows. A concurrent worker skips locked rows instead of waiting. The same statement updates them to running with that worker's id, so when the lock is released the rows are no longer pending and no other claim query can match them.

Q7. What does SKIP LOCKED actually do? Why not just FOR UPDATE?

FOR UPDATE makes a second transaction wait for the lock on the same row. Every worker would pile up behind the top-priority row and run one at a time. SKIP LOCKED leaves out rows it can't lock right away, so workers spread across different rows. The cost is an inconsistent view: you might not see a row that's locked by a transaction that later rolls back.

Q8. Is there a race between the SELECT and the UPDATE in claim_jobs?

No. Both are in one statement (data-modifying CTE), so one transaction. The SELECT takes row locks, and the UPDATE runs while those locks are held. Nobody else can change those rows between the two.

Q9. What isolation level does this rely on?

Default READ COMMITTED. Row locks give the exclusivity, not snapshot isolation. Under REPEATABLE READ or SERIALIZABLE you'd get serialization failures on concurrent updates and would need retry logic.

Q10. Can a deadlock happen?

In the claim path it's unlikely: each statement locks rows in one index order and skips any it can't get, so it never waits. Deadlocks are more plausible where multi-row UPDATEs (recovery, complete_jobs, renew_leases) touch overlapping rows in different orders. Postgres detects deadlocks and aborts one transaction. Ordering IDs before an UPDATE reduces the risk.

Q11. How is priority handled, and is it strict?

ORDER BY priority, scheduled_at at claim time, supported by the partial index. It is **not** strict globally: SKIP LOCKED can skip a locked high-priority row, and jobs in a batch run concurrently. Constant high-priority traffic can also starve low-priority jobs. Fixes: aging (raise priority over time), weighted fair queues, or separate queues with dedicated workers.

Q12. What about FIFO ordering?

Best effort only. Strict FIFO needs one consumer or per-key ordering (like Kafka partitions). You could get per-key ordering by claiming only the oldest pending job per key, at the cost of parallelism.

C. Reliability, leases and failure handling

Q13. What happens when a worker crashes mid-job?

Its lease stops being renewed. After at most lease_duration plus recovery_interval, any worker's recovery loop sets the job back to pending, and the next claim increments attempt_count. The SIGKILL test shows recovery in 5.1 s with a 6 s lease.

Q14. Why leases instead of keeping the transaction open?

An open transaction pins a connection per running job, holds back vacuum (old xmin), and hides progress. With leases, ownership is data, connections go back to the pool immediately, and you can inspect it with SQL.

Q15. How did you choose lease and renewal times?

Renewal interval is about a third of the lease (10 s / 30 s), so a worker can miss two renewals before losing the job. Shorter means faster recovery but more false recoveries during pauses. Longer means fewer duplicates but slower recovery. Tune it from p99 pause and network times.

Q16. What if a worker is alive but paused longer than the lease (GC, VM stall)?

The job is recovered and runs on another worker. When the paused worker wakes up, its writes are rejected: complete, fail and renew all require worker_id = me AND status = 'running', and 0 rows updated means 'lease lost'. Its next renewal fails, so it cancels that handler. I found this gap in a self-review and added a test for it. The remaining hole is the same worker id reclaiming its own job, which a full fencing token (a claim_version that increments per claim) would close. This is Kleppmann's point that distributed locks need fencing. One thing a check can't do is undo a side effect the paused worker already performed, so handlers still need to be idempotent.

Q17. What happens if the database goes down?

Workers' calls raise errors, the poll loop logs them and sleeps 2 s, then retries. Jobs are durable, so nothing committed is lost. In-flight results that weren't committed get re-run after leases expire, which is at-least-once again. With HA (Patroni/RDS), the gap is the failover time.

Q18. How do retries work? What about backoff?

On failure with attempts left: pending + scheduled_at = now + retry_delay_sec. The claim query ignores future-scheduled jobs. It's a fixed delay today. I'd add exponential backoff with jitter to avoid retry storms during a downstream outage.

Q19. What's a dead-letter queue for, and how do you handle it?

A place for jobs that have used every attempt, so they stop using capacity and stay available for inspection. You alert on DLQ growth, then fix the bug and requeue (UPDATE status='pending', attempt_count=0) or discard.

Q20. What's a poison pill and does your system handle it?

A job that always fails or crashes its worker. Exceptions are handled through max_attempts. A job that crashes the process used to loop forever, because recovery didn't check attempt_count. Now recovery dead-letters a job whose attempts are used up and logs a dead_lettered event, and the poison_pill test checks this.

Q21. How do heartbeats differ from leases?

Heartbeats track *worker* liveness for monitoring (active_workers, alerts). Leases track *job* ownership and drive recovery. Keeping them separate means a worker can be alive while one of its jobs has lost its lease.

Q22. Why does every worker run recovery? Isn't that duplicated work?

It removes the single point of failure: no leader election is needed. It is safe because recovery is one conditional UPDATE and row locks serialize concurrent runs. The cost is redundant queries. With many workers you'd add jitter, a lower frequency, or use pg_try_advisory_lock so only one runs at a time.

D. Delivery semantics and idempotency

Q23. Is it exactly-once?

No: at-least-once execution with exclusive claiming. A worker can perform the side effect and crash before committing 'completed', and then the job runs again. Exactly-once needs the side effect and the completion record to commit atomically, which only works when the effect is in the same database, or when the handler is idempotent.

Q24. How would you make a payment or email handler safe?

Use an idempotency key derived from the job id. Pass it to the payment provider's Idempotency-Key header, or record 'sent' in a dedup table in the same transaction that marks the job complete. Upserts instead of inserts. Check before acting where the downstream system supports it.

Q25. Explain the idempotency key on enqueue.

A UNIQUE column with INSERT ... ON CONFLICT, so retried submissions return the same job id. RETURNING (xmax = 0) tells whether the row was new, so the enqueued event is logged once. It prevents duplicate *jobs*, not duplicate *execution*.

Q26. What is the xmax trick?

xmax is a system column. For a freshly inserted row version it is 0. With ON CONFLICT DO UPDATE the updated row has xmax set to the current transaction id. It's a common idiom, but it depends on Postgres internals rather than a documented API.

Q27. How do you detect duplicate completions?

job_events is append-only. After each benchmark run, group 'completed' events by job_id HAVING COUNT > 1. Limitation: it catches double *recording*, not a side effect that ran twice before a crash.

Q28. How would you enqueue a job atomically with a business write?

That's the main advantage of a Postgres queue: INSERT the order and INSERT the job in the same transaction (transactional outbox). If the transaction rolls back, there's no job. With an external broker you need an outbox table plus a relay, or CDC (Debezium).

E. PostgreSQL internals

Q29. Why a partial index for claiming?

Only pending rows matter for claims. A partial index WHERE status='pending' stays small and hot in cache, and its column order (queue_name, priority, scheduled_at) matches the filter and ORDER BY, so Postgres walks the index and stops at LIMIT without a sort.

Q30. What is MVCC bloat and how does it affect a queue?

Every UPDATE writes a new row version and leaves the old one dead until VACUUM. A queue updates each row several times (pending, running, completed), so dead tuples build up quickly. Index entries pointing to dead versions slow down scans. Mitigations: aggressive per-table autovacuum, HOT updates (fillfactor < 100, avoid updating indexed columns), partitioning, and deleting or archiving finished jobs.

Q31. How would you verify your hypothesis about the 100k slowdown?

Run EXPLAIN (ANALYZE, BUFFERS) on the claim query early and late in a run and compare buffers read. Watch n_dead_tup and last_autovacuum in pg_stat_user_tables. Rerun with VACUUM between phases or with autovacuum_vacuum_scale_factor=0.01 and see if the gap closes.

Q32. Why use data-modifying CTEs?

They let one statement lock, update, insert audit rows and return results atomically. That's one round trip, which is where most of the 16x came from. All sub-statements see the same snapshot, and a CTE that writes runs even if the outer query doesn't reference it (the 'ev' CTE).

Q33. Why is NOW() used instead of client time? Any gotcha?

So there's one clock and no skew between worker hosts. Gotcha: NOW() is the *transaction* start time. For long transactions use clock_timestamp().

Q34. What about connection limits?

Each process has an asyncpg pool (max 10). Total connections = processes x pool size. Postgres connections are processes (about 5-10 MB each), so the next step is PgBouncer in transaction mode, which works here because every operation is one statement and nothing uses LISTEN or session state.

Q35. What does fsync/synchronous_commit have to do with your numbers?

Both stayed on, so each commit waits for the WAL flush. That makes the numbers honest for durable settings. Turning synchronous_commit off would raise throughput, at the risk of losing the last few hundred ms of commits on a crash.

F. Performance and benchmarking

Q36. Tell me about an optimization you made.

STAR format. Situation: 1.1k jobs/s with 16 workers. Task: find the bottleneck. Action: counted round trips (about 9 per job), moved claim, start, events, completion and stats into two batched SQL functions, and ran handlers concurrently. Result: 17.8k sustained, 16x, zero duplicates, verified twice.

Q37. How did you know it was round trips and not locks?

One worker had no contention and still ran at only 162 jobs/s, about 6 ms per job, which is roughly 9 sequential sub-millisecond queries. Adding workers scaled throughput nearly linearly at first, which wouldn't happen if lock contention were the limit.

Q38. How do you know Python isn't the bottleneck now?

4 processes x 4 workers gave the same throughput as 1 x 16, and Postgres CPU was about 470%. If the GIL or the event loop were the limit, spreading across processes would have helped.

Q39. What are the limits of your benchmark?

No-op handlers (pure overhead), one machine (no network latency), Postgres in Docker on a laptop, two runs, one queue. A realistic benchmark would use I/O-bound handlers with a latency distribution, workers on separate hosts, longer runs, and a report of percentiles of end-to-end latency, not only throughput.

Q40. Why does batch size trade latency for throughput?

One round trip covers N jobs, but every job in the batch waits for the slowest before completion is recorded and the leases released. Large batches can also hoard work when the queue is short, leaving other workers idle.

Q41. How would you measure end-to-end latency?

completed_at - created_at per job (wait + exec), reported as percentiles. The exporter already reports avg wait (claimed_at - created_at) and exec p95/p99. I'd add a histogram for queue wait.

G. Scaling and system design extensions

Q42. How would you scale to 10x throughput?

First, cheap steps: bigger batches, PgBouncer, per-job completion, autovacuum tuning, retention. Then partition jobs by queue_name and/or move hot queues to a dedicated Postgres. Beyond a single primary's write capacity, use SQS, Kafka or Redis Streams through the backend interface.

Q43. When would you NOT use Postgres as a queue?

When you need fan-out to many consumers, replay or retention of a stream, tens of thousands of messages per second with real payloads, or isolation from the OLTP database. Also when the team already runs a managed broker like SQS.

Q44. Compare with SQS / Kafka / Redis / RabbitMQ.

SQS: managed, visibility timeout (same idea as a lease), at-least-once, no ordering on standard queues. Kafka: a partitioned log, ordered per partition, replay, consumer groups, parallelism capped by partition count, heavier to run. Redis: very fast, streams with consumer groups and XPENDING/XCLAIM (lease-like), weaker durability. RabbitMQ: push-based with acks and prefetch, real routing. All are at-least-once at best.

Q45. How would you add scheduled / cron jobs?

Delayed jobs already work through scheduled_at. For cron: a scheduler process (or leader-elected worker via advisory lock) that enqueues the next run with an idempotency key like 'cron:name:2026-10-09T10:00', so duplicate schedulers can't double-enqueue.

Q46. How would you add rate limiting per job type or tenant?

Token buckets in a table updated atomically, or count running jobs per key at claim time (a WHERE clause or NOT EXISTS against running jobs), or separate queues with a capped number of workers each.

Q47. How would you support job dependencies (DAGs)?

A dependencies table and a 'blocked' status. When a job completes, unblock children whose parents are all complete, in the same transaction as the completion.

Q48. How do you deploy workers?

Stateless containers (Docker Compose here; Kubernetes Deployment in prod). Scale horizontally on queue depth (KEDA on jobqueue_pending_total). Graceful SIGTERM drain with terminationGracePeriod longer than the typical job time.

Q49. How would you handle multi-tenancy fairness?

Round-robin claiming across tenants: rank pending jobs per tenant with a window function, or keep per-tenant queues and have workers rotate through them. Add per-tenant concurrency caps.

H. Observability and operations

Q50. What would you alert on, and why those?

Queue depth (saturation), failure rate (errors), zero active workers (outage), DLQ growth (systematic bug), p99 exec latency (degraded dependencies). Each one points to an action someone can take.

Q51. Problem with your metrics implementation?

The exporter runs COUNT and PERCENTILE_CONT over the table every 15 s, which gets expensive as history grows. The '_total' metrics are current row counts, not monotonic counters, so archiving breaks rate(). I'd keep counters in-process or as summary tables.

Q52. How would you debug 'job X ran three times'?

SELECT * FROM job_events WHERE job_id = X ORDER BY occurred_at: claimed/started by which workers, retried with which errors, and lease_expired events if a worker was lost. Check last_error, then the workers table for heartbeats around those times.

Q53. Queue depth is growing. What do you check?

Throughput vs arrival rate, active workers, failure/retry rate (retries add load), handler latency (slow dependency?), DB CPU, locks and connection saturation, and whether one job type is hogging batches. Then scale workers or fix the slow dependency.

I. Testing

Q54. How do you test a distributed system like this?

Tests built around failure scenarios against a real Postgres: concurrent claiming, retries, DLQ, lease renewal, and a real SIGKILL of a separate OS process. Everything runs in CI on every push. Next would be fault injection (network partitions with toxiproxy, paused processes with SIGSTOP), property-based tests on the state machine, and Jepsen-style history checking.

Q55. Why kill a separate process instead of cancelling a task?

Cancellation runs cleanup handlers. SIGKILL can't be caught, which is like an OOM kill or a lost node. The test proves recovery doesn't depend on the dying worker doing anything.

Q56. How do you test the stalled (zombie) worker case?

The zombie_fencing test simulates it deterministically: claim with a 1 s lease, let it expire, recover, re-claim as another worker, then try renew, complete and fail as the stale worker and assert all three are rejected and the owner's result is stored once. A more realistic version would SIGSTOP a real worker process past its lease, then SIGCONT it.

Q57. Are your tests flaky? How did you deal with timing?

They poll for state with deadlines instead of sleeping fixed amounts, use short leases in the crash test, and isolate each scenario in its own queue_name. Any test that depends on time can still flake on a slow CI runner, so the deadlines leave plenty of room.

J. Code and design quality

Q58. Why the AbstractQueueBackend interface?

So workers depend on operations (claim, complete, renew, recover) and not on SQL, which leaves a way to move to SQS or Kafka. Honestly, the interface still reflects the old single-job API. The batch methods should be added to it.

Q59. Why put logic in stored functions rather than Python?

Atomicity and a single round trip. Downside: harder to test and version. I'd add a migrations tool and SQL-level tests.

Q60. How is configuration managed?

Dataclasses with environment overrides. It's a module-level singleton, which the tests modify directly (for example a short lease). That works, but explicit dependency injection would be cleaner.

Q61. Any security concerns?

Tests and benchmark build SQL with f-strings (test-only, but should use parameters). Default credentials in compose. Payloads are JSONB, so never put secrets in them, or encrypt them. Workers run any registered handler, so validate job_type against an allowlist (already done by handler lookup).

K. Behavioral angles on this project

Q62. Tell me about a time you were wrong.

Early docs claimed 'exactly-once'. Once I understood the crash-after-side-effect window I rewrote the README around at-least-once and added a precise definition of what 'zero duplicates' measures. Later a self-review found a related bug: a stalled worker could overwrite a reclaimed job. I added ownership checks and four new failure tests, and corrected scale_story.md, which still claimed exactly-once.

Q63. How did you decide what not to build?

I scoped it to coordination mechanics and listed non-goals (fan-out, replay, DAGs). The README has a 'Current scope' section that is honest about future work.

Q64. How did you make sure your claims were credible?

Two full benchmark runs per configuration, hardware and durability settings published, a reproduce command, duplicate-check method explained, hypotheses labelled 'not yet verified'.

13. Cheat sheet

Thing	Value / fact
Claim primitive	SELECT ... ORDER BY priority, scheduled_at LIMIT n FOR UPDATE SKIP LOCKED, in a CTE with UPDATE + event INSERT
Delivery semantics	At-least-once execution, exclusive claiming of available jobs
Lease / renewal / recovery	30 s / 10 s / 15 s (test: 6 s / 2 s / 1 s)
Retry	Fixed retry_delay_sec through scheduled_at, default 3 attempts, then failed, then dead_letter
Batch size	Default 10, benchmark best 50
Before / after	9 round trips per job -> 2 per batch. 1,116 -> 17,800 jobs/s (16x). 1 worker 162 -> 470
Bulk enqueue	37-40k/s via unnest + ON CONFLICT DO NOTHING
Claim latency	p50 1.05 ms, p99 1.32-1.45 ms
Bottleneck now	Postgres CPU (~470%). Same result with 1 vs 4 processes
Unverified hypothesis	100k run 30% slower: dead index entries / autovacuum lag
Crash test	SIGKILL separate process, recovered in 5-6 s, attempt_count == 2
Test suite	10 scenarios in chaos_test.py (incl. fencing, poison pill, lost lease, batch lease hold), GitHub Actions + postgres:16 service
Hardening done	Ownership checks on complete/fail/renew; lost lease cancels handler; recovery dead-letters poison pills; atomic fail_job
Top 3 open gaps	Same-worker reclaim (needs claim_version token); whole-batch latency coupling; no retention / bloat control
Top 3 next steps	Fencing token + per-job completion; LISTEN/NOTIFY + backoff with jitter; retention + autovacuum tuning

Lines worth memorizing

"Exclusive claiming isn't exactly-once execution."
"The lock protects the claim. The lease protects ownership. The ownership check protects the result."
"The first version was limited by round trips, not by locking."
"I use Postgres as a queue when transactional enqueue and operational simplicity matter more than raw throughput."